M09 # Slash Commands

Méthodologie Intermediate

Skills déclenchés manuellement via /nom (CC 2.1.3)

CC 2.1.3 : Les skills utilisent `.claude/skills/<name>/SKILL.md` ; `disable-model-invocation: true` impose une invocation manuelle. Les anciens fichiers `.claude/commands/*.md` restent pris en charge. La syntaxe `/nom` et le comportement sont identiques.

Principe

Une slash command est un fichier Markdown qui définit un workflow. Claude l'exécute comme s'il avait reçu ce texte en prompt, avec la possibilité de passer des arguments dynamiques.

Emplacement des fichiers

```
.claude/skills/          # Skills du projet
├── commit/SKILL.md      → /commit
├── review-pr/SKILL.md   → /review-pr
└── deploy/SKILL.md      → /deploy

~/.claude/skills/        # Skills personnels
├── release/SKILL.md     → /release
└── sync/SKILL.md        → /sync
```

Les commandes globales sont disponibles dans toutes les sessions, quel que soit le projet.

Invocation et arguments

```
/commit                  # Aucun argument
/deploy production       # Argument positionnel
/release minor           # Bump de version
```

Dans le fichier Markdown, les arguments sont accessibles via des variables :

```
Déployer en environment : $ARGUMENTS[0]
Avec la stratégie : $ARGUMENTS[1]

Si aucun argument fourni : utiliser "staging" par défaut.
```

La syntaxe `$0`, `$1` est équivalente à `$ARGUMENTS[0]`, `$ARGUMENTS[1]`. L'ancienne notation avec point (`$ARGUMENTS.0`) est dépréciée depuis v2.1.19.

Ajouter `argument-hint` dans le frontmatter pour afficher un placeholder dans le menu : `argument-hint: "<env> [--dry-run]"` → le picker affiche `/deploy <env> [--dry-run]`.

Structure recommandée

```
# Nom de la Commande
## Objectif
Ce que fait cette commande en une phrase.
## Processus
1. **Étape 1** — instructions détaillées
2. **Étape 2** — instructions détaillées
3. **Étape 3** — instructions détaillées
## Arguments
- $0 : environnement cible (défaut: staging)
## Format de sortie
Description du résultat attendu.
```

Commandes natives récentes

Ces deux entrées sont des commandes internes. `/simplify`, `/batch`, `/loop` et `/debug` sont des skills intégrés qui fonctionnent comme les vôtres.

Commande	Usage
`/goal [condition]`	Mode autonome multi-tours : Claude travaille jusqu'à ce que la condition soit remplie, avec overlay temps/tours/tokens
`/scroll-speed`	Curseur interactif pour régler la vitesse de défilement de la souris avec aperçu en direct

Commandes utiles à créer

Commande	Usage
`/commit`	Commit conventionnel automatique
`/review-pr`	Revue de PR selon critères projet
`/release [patch|minor|major]`	Bump version + changelog
`/sync`	Vérification cohérence multi-fichiers
`/security-check`	Scan rapide de la config

Command vs skill vs agent

Mécanisme	Quand l'utiliser
Command	Workflow ponctuel, procédure à suivre
Skill	Connaissance réutilisable + ressources embarquées
Agent	Spécialiste récurrent avec mémoire propre

Un répertoire de skill peut inclure fichiers de référence et scripts, que l'invocation vienne du modèle ou de l'utilisateur. La mémoire d'agent est un mécanisme persistant distinct.

Conseils pratiques

Une commande, une responsabilité. Séparer les procédures distinctes. Nommer par un verbe d'action (`commit`, `release`, `sync`) pour rendre l'intention lisible.